

Cuando la IA Programa: El Rol de las Especificaciones en el Futuro del Desarrollo de Software

Maximiliano Exequiel Rodríguez Martínez¹.

¹ Escuela de Ciencias Tecnológicas, Universidad Nacional del Oeste, Argentina
maxi.rodriguez.3105@gmail.com

Resumen. La aparición de herramientas capaces de generar código de manera automática puso en evidencia un problema que ya venía creciendo: el *vibe coding*. Es decir, resultados que parecen correctos a simple vista, pero que no siguen un criterio claro, mezclan estilos y dejan poca trazabilidad entre lo que se pidió y lo que se implementó. Esto termina generando ambigüedad, inconsistencias y dificultades para reproducir el trabajo. En respuesta a este escenario, Spec-Driven Development (SDD) propone volver a poner la especificación en el centro del proceso. No sólo para describir el comportamiento esperado del sistema, sino también para indicar cómo deben trabajar los agentes —humanos o de IA (Principalmente no humanos)— que participan en su construcción: qué rol asumir, qué supuestos respetar y cómo producir, validar y documentar cada artefacto. El trabajo explora cómo este tipo de especificaciones operativas puede aportar orden y coherencia en proyectos donde intervienen modelos generativos. Se presenta OpenSpec como ejemplo concreto, mostrando cómo un formato de este tipo permite definir roles, criterios de diseño y estrategias de validación para agentes de IA. También se discute el ciclo Specify–Plan–Task–Implement y su utilidad para mantener una relación clara entre intención y resultado. Finalmente, se analiza cómo la precisión de estas instrucciones influye en la calidad del software generado y se argumenta que SDD ofrece un camino para trabajar con mayor disciplina y transparencia en equipos híbridos humano–IA, evitando el *vibe coding* y favoreciendo prácticas más estables y reproducibles.

Palabras Clave: Especificaciones, Inteligencia Artificial, Spec-Driven Development, OpenSpec.

1. Introducción

La adopción masiva de modelos de lenguaje de gran escala (LLMs) para la generación de código ha transformado el proceso de desarrollo de software. Según un informe de Harness Inc. basado en una encuesta a 900 ingenieros en cuatro países, más del 70% de los equipos ya integra herramientas de IA en alguna fase del ciclo de desarrollo [1]. Sin embargo, esta aceleración no ha venido acompañada de un refuerzo equivalente en las prácticas de especificación de requisitos y diseño.

El resultado más visible de esta brecha es el *vibe coding*, término acuñado por Andrej Karpathy en febrero de 2025 para describir la práctica de generar código respondiendo a indicaciones informales, sin un marco de requisitos previo y aceptando el resultado sin comprenderlo en profundidad [2]. Aunque el fenómeno tomó nombre recientemente, sus consecuencias —código ambiguo, difícil de mantener y con vulnerabilidades de seguridad— son conocidas en la ingeniería de software desde hace décadas.

Frente a este escenario, la comunidad técnica ha comenzado a recuperar y reinterpretar una idea que tiene raíces profundas en la disciplina: la especificación como artefacto primario del desarrollo. Este conjunto de prácticas, agrupado bajo el nombre de Spec-Driven Development (SDD), propone que la precisión de las instrucciones que reciben los agentes —ya sean humanos o sistemas de IA— determina directamente la calidad del software resultante.

Este trabajo tiene dos objetivos. Primero, caracterizar el problema del *vibe coding* y sus consecuencias en términos de calidad y seguridad. Segundo, presentar SDD como respuesta metodológica, analizando sus principios, su relación con enfoques preexistentes y su aplicación práctica mediante herramientas como OpenSpec [3]. La contribución principal es una discusión articulada sobre cómo las especificaciones operativas pueden estructurar el trabajo en equipos híbridos humano-IA, preservando trazabilidad y reproducibilidad.

2. Marco Conceptual

La idea de formalizar el comportamiento esperado de un sistema antes de su implementación no es nueva. En 1992, Bertrand Meyer formuló el concepto de *Design by Contract* (DbC), según el cual cada componente de software define explícitamente sus precondiciones, postcondiciones e invariantes [4]. Este enfoque estableció que la especificación no es documentación opcional, sino una condición de corrección del software. SDD retoma esta intuición y la extiende al contexto de los agentes de IA.

En la misma línea, el Desarrollo Guiado por Pruebas (TDD) [5] y el Desarrollo Guiado por Comportamiento (BDD) [6] proponen que los criterios de aceptación preceden al código. Sin embargo, ambos enfoques operan principalmente en el nivel de verificación y validación, no en el nivel de instrucción a agentes generativos. SDD ocupa ese espacio anterior: define el *qué* y el *por qué* con suficiente detalle técnico como para que un agente de IA pueda actuar sobre esa definición sin ambigüedad [7].

En cuanto al fenómeno específico del *vibe coding*, la primera revisión sistemática comprensiva fue publicada por Y. Ge y colaboradores [2], quienes analizaron más de 1.000 trabajos académicos para caracterizar el ecosistema de generación de código

con LLMs, sus entornos, mecanismos de retroalimentación y riesgos. Este trabajo constituye la referencia académica de base para entender la magnitud del problema que SDD busca resolver.

3. El Problema del Vibe Coding

El *vibe coding* se caracteriza por la generación de código que responde más a patrones estadísticos que a una intención clara y verificable. Y. Ge y colaboradores [2] identifican como factores determinantes la ausencia de especificaciones formales previas, la dependencia de prompts informales y la falta de mecanismos de retroalimentación estructurada entre la intención del desarrollador y el artefacto generado.

Las consecuencias son concretas. Desde el punto de vista de la seguridad, investigadores de Wiz Research encontraron que una de cada cinco organizaciones que adoptaron herramientas de *vibe coding* presentaba exposiciones sistémicas a riesgos críticos, incluyendo claves de API expuestas en repositorios, ausencia de validación de entrada y configuraciones inseguras en servicios de autenticación [8]. Nagli y Schindel documentan que estos riesgos no son accidentales sino estructurales: emergen de la ausencia de criterios de diseño de seguridad en las instrucciones que reciben los modelos.

Desde el punto de vista de la mantenibilidad, el informe de Harness [1] señala que los equipos que no establecen especificaciones previas al uso de IA reportan un 40% más de tiempo en revisión y corrección de código generado. La aceleración inicial que prometen estas herramientas se diluye en deuda técnica acumulada.

En síntesis, el *vibe coding* no es simplemente una práctica descuidada: es el resultado predecible de incorporar agentes generativos en un proceso de desarrollo que no fue rediseñado para recibirlos.

4. Spec-Driven Development

4.1 Principios

Spec-Driven Development (SDD) propone que la especificación precede y guía el código, y que esta relación debe mantenerse de forma explícita y versionada a lo largo del ciclo de vida del proyecto [7]. A diferencia de los enfoques de documentación tradicional, SDD no concibe la especificación como un artefacto estático que se abandona una vez que comienza la implementación, sino como la fuente de verdad que evoluciona junto al sistema.

Böckeler [7] propone una taxonomía de tres niveles de adopción:

- Spec-first: la especificación se escribe antes de codificar y se descarta una vez implementada.
- Spec-anchored: la especificación se mantiene como referencia durante la evolución del sistema.
- Spec-as-source: la especificación es el artefacto primario; el código se regenera a partir de ella.

El tercer nivel, aunque actualmente el más exigente en términos de disciplina de equipo, es el que mejor aprovecha las capacidades de los agentes de IA: un modelo puede generar o regenerar código a partir de una especificación precisa con mayor consistencia de lo que puede hacerlo a partir de una descripción informal.

En el ámbito de la ingeniería de requisitos, este principio se vincula directamente con dos prácticas esenciales: la trazabilidad y la gestión de cambios. Sommerville define la administración de requerimientos como "el proceso de comprender y controlar los cambios en los requerimientos del sistema" [9, p. 112], y establece que cada requerimiento debe identificarse de forma exclusiva de modo que "pueda tener referencia cruzada con otros requerimientos y usarse en las evaluaciones de seguimiento" [9, p. 113]. Estas dos condiciones, trazabilidad de cada requisito hacia el código y las pruebas que lo implementan, y gestión explícita de su evolución son precisamente las que SDD refuerza al mantener la especificación como artefacto vivo y versionado, asegurando que las decisiones de diseño y los artefactos generados por agentes de IA estén siempre anclados en criterios explícitos y verificables.

4.2 Relación con Enfoques Preexistentes

SDD no reemplaza a TDD ni a BDD; los complementa en capas distintas del proceso:

- SDD define la intención técnica, los roles de los agentes y los criterios de diseño.
- BDD valida que el comportamiento del sistema coincide con los escenarios acordados con los interesados.
- TDD verifica la corrección del código a nivel de unidades.

La relación con *Design by Contract* [4] es de inspiración directa: SDD extiende la lógica de precondiciones y postcondiciones desde el nivel de componentes individuales hacia el nivel del proceso completo de construcción del software.

5. OpenSpec como Caso Ilustrativo

OpenSpec es una herramienta de código abierto desarrollada por Fission AI que implementa algunos principios de SDD en el contexto de agentes de IA modernos [3]. No se presenta aquí como referencia metodológica consolidada —es un proyecto joven sin validación académica formal— sino como ejemplo concreto de cómo estos principios pueden materializarse en una herramienta de uso cotidiano.

Su mecanismo central es la generación de un documento de especificación antes de que se escriba cualquier línea de código. Este documento incluye el alcance del sistema (en proyectos de desarrollo) o el alcance del cambio (en actividades de mantenimiento), junto con los requisitos funcionales y no funcionales que definen qué debe hacer el sistema y bajo qué condiciones de calidad o limitaciones. También detalla los supuestos de diseño, las interfaces afectadas y las fases de implementación planificadas. Los agentes de IA que trabajan sobre el repositorio reciben este documento como contexto estructurado, en lugar de operar sobre prompts informales.

OpenSpec tiene soporte nativo para más de 20 herramientas de codificación asistida por IA, incluyendo Claude Code, Cursor y GitHub Copilot [3], lo que ilustra la dirección hacia la que se mueve el ecosistema: hacia flujos de trabajo donde la

especificación es el punto de partida compartido entre agentes humanos y no humanos.

6. El Ciclo Specify–Plan–Task–Implement

El flujo de trabajo de OpenSpec articula cuatro etapas que sintetizan la dinámica operativa de SDD [3]:

- Specify: el agente humano describe el cambio deseado en términos de intención (requisitos) y criterios de diseño, no en términos de implementación.
- Plan: el sistema genera una propuesta que incluye decisiones de diseño, interfaces afectadas y tareas de implementación. El equipo revisa y refina este plan antes de escribir código.
- Task: el cambio se descompone en unidades de trabajo ejecutables, cada una trazable hacia el documento de especificación.
- Implement: el agente de IA construye el artefacto siguiendo el plan aprobado, con cada decisión anclada en la intención declarada.

El valor central de este ciclo es la revisión explícita entre la etapa de planificación y la de ejecución: la especificación se convierte en un contrato que el equipo valida antes de delegar trabajo al agente, reduciendo la brecha entre requisitos e implementación en código..

7. Impacto en la Calidad del Software

La relación entre precisión de especificaciones y calidad del software generado tiene sustento empírico. El informe de Harness [1] documenta que los equipos que establecen requisitos y criterios de diseño explícitos antes de invocar herramientas de IA reportan menor tiempo de revisión, menor frecuencia de regresiones y mayor confianza en los artefactos generados.

Desde la perspectiva de seguridad, los riesgos documentados por Nagli y Schindel [8] —exposición de credenciales, ausencia de validación de entrada, configuraciones inseguras— son precisamente el tipo de problemas que una especificación con requisitos de seguridad explícitos puede prevenir. Cuando el agente recibe instrucciones que incluyen consideraciones de seguridad como parte del contrato de diseño, la probabilidad de introducir errores en el código generado disminuye de forma significativa.

En escenarios de colaboración híbrida entre humanos e inteligencia artificial, SDD refuerza además la calidad del proceso. Hadad, Doorn y Leite establecen que "la calidad de los proyectos de software es en gran parte dependiente de la calidad de sus procesos de requisitos" [10, p. 63], premisa que en contextos donde intervienen agentes de IA cobra mayor relevancia aún: la especificación funciona como un artefacto compartido y persistente que asegura que distintos participantes —ya sean desarrolladores humanos o modelos de IA— puedan retomar el proceso sin pérdida de contexto ni necesidad de reconstruir los requisitos iniciales.

8. Conclusiones y Trabajos Futuros

El *vibe coding* es el síntoma de un proceso de desarrollo que no fue adaptado para incorporar agentes generativos. Las consecuencias —ambigüedad, inconsistencias, riesgos de seguridad y deuda técnica— son predecibles y documentadas [1, 2, 8]. Spec-Driven Development ofrece una respuesta que no requiere abandonar las herramientas de IA, sino disciplinar la forma en que se las instruye.

Los principios de SDD tienen raíces en enfoques consolidados —Design by Contract [4], BDD, TDD— y su convergencia con el paradigma de agentes de IA es natural: un agente que recibe una especificación precisa produce resultados más coherentes y verificables que uno que opera sobre prompts informales.

Como limitación del presente trabajo, se señala que el análisis es de naturaleza conceptual y no incluye una evaluación empírica de SDD en entornos reales. Como trabajos futuros se propone: (a) diseñar métricas de trazabilidad específicas para evaluar la correspondencia entre especificaciones y artefactos generados; (b) conducir estudios comparativos entre equipos que adoptan SDD y equipos que no lo hacen, midiendo calidad, seguridad y tiempo de revisión; y (c) analizar la adopción de SDD en proyectos de código abierto donde la colaboración con agentes de IA es ya una práctica documentada.

Referencias

1. Harness Inc. (2025). The State of AI in Software Engineering. Informe encargado a Coleman Parkes Research (n=900). Harness Inc., Estados Unidos.
2. Ge, Y., Mei, L., Duan, Z., Li, T., Zheng, Y., Wang, Y., et al. (2025). A Survey of Vibe Coding with Large Language Models. arXiv:2510.12399. Disponible en: <https://arxiv.org/abs/2510.12399>
3. Fission AI. (2025). OpenSpec: A Lightweight Spec-Driven Framework for AI Coding Agents. openspec.dev. Disponible en: <https://openspec.dev/>.
4. Meyer, B. (1992). Applying "Design by Contract". IEEE Computer, 25(10), 40–51.
5. Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley Professional.
6. North, D. (2006). *Introducing BDD*. Disponible en: <https://dannorth.net/blog/introducing-bdd/>
7. Böckeler, B. (2025). Understanding Spec-Driven-Development: Kiro, spec-kit, and TESSL. Thoughtworks Blog.
8. Nagli, G., Schindel, A. (2025). Common Security Risks in Vibe-Coded Apps. Wiz Research, Wiz Inc., 18 de septiembre de 2025. Disponible en: <https://www.wiz.io/blog/common-security-risks-in-vibe-coded-apps>
9. Sommerville, I. (2011). *Ingeniería de Software* (9ª ed.). Pearson Educación.
10. Hadad, G. D. S., Doorn, J. H., y Sampaio do Prado Leite, J. C (2025). *Gestión de Requisitos para la Ingeniería de Software*. Alfaomega.